

Introduction to Algorithm

기초 알고리즘

Merge Sort와 Quick Sort

퀵 정렬



정렬 (Sorting)

Q. 정렬이 필요한 이유?

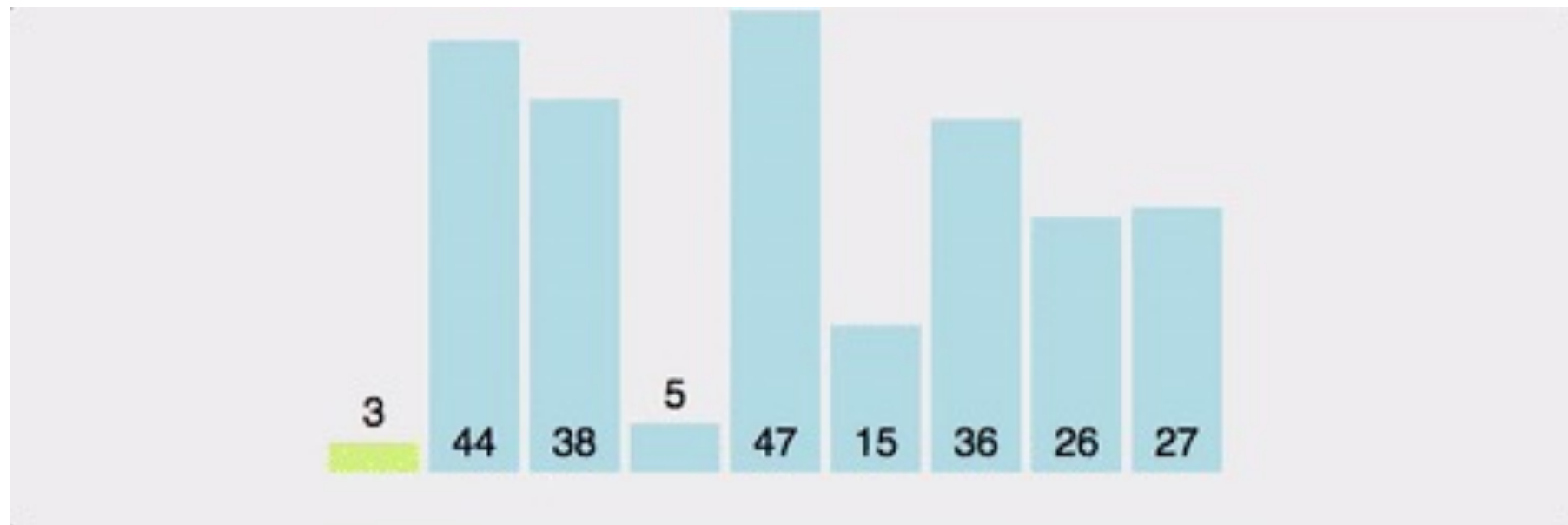
A. 오름차순 및 내림차순으로 정렬되어 있다면 특정 원소를 좀 더 효율적으로 찾을 수 있음

이름	최선	평균	최악	평균 공간 복잡도
삽입 정렬 (Insertion Sort)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
버블 정렬 (Bubble Sort)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
선택 정렬 (Selection Sort)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
합병 정렬(Merge Sort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
퀵 정렬(Quick Sort)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
힙 정렬(Heap Sort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

특정 정렬이 빠르다고 항상 좋은 것은 아님
데이터의 특성, 크기에 따라 적절한 정렬 방법을 사용해야 함



퀵 정렬 (Quick Sort)



- 배열의 요소들 중에서 **피벗**(Pivot)을 정하여, 피벗의 앞에는 피벗보다 작은 원소들이 오고, 피벗 뒤에는 피벗보다 큰 값이 오도록 배열을 둘로 나눔
- 분할된 두 개의 배열의 크기가 0이나 1이 될 때까지, 분할된 두 배열에 대해 **재귀적으로** 이 과정을 반복
- 재귀 호출이 한 번 진행될 때마다 최소한 하나의 원소가 최종적인 위치에 있게 되므로, 종료됨이 보장



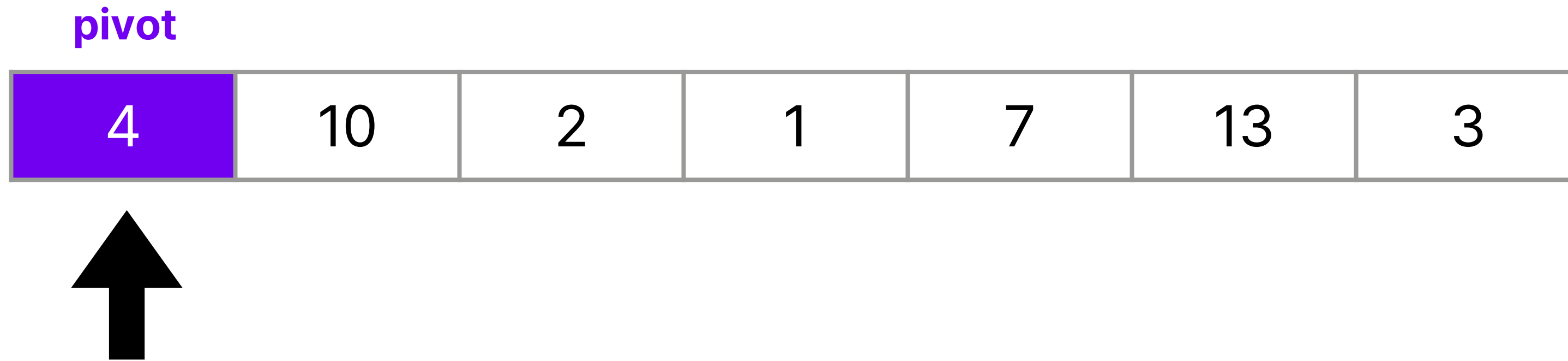
퀵 정렬 (Quick Sort)

4	10	2	1	7	13	3
---	----	---	---	---	----	---

퀵 정렬(Quick sort)은 주어진 리스트를 빠르게 정렬하는 알고리즘으로,
분할정복 기반의 문제 해결 방법



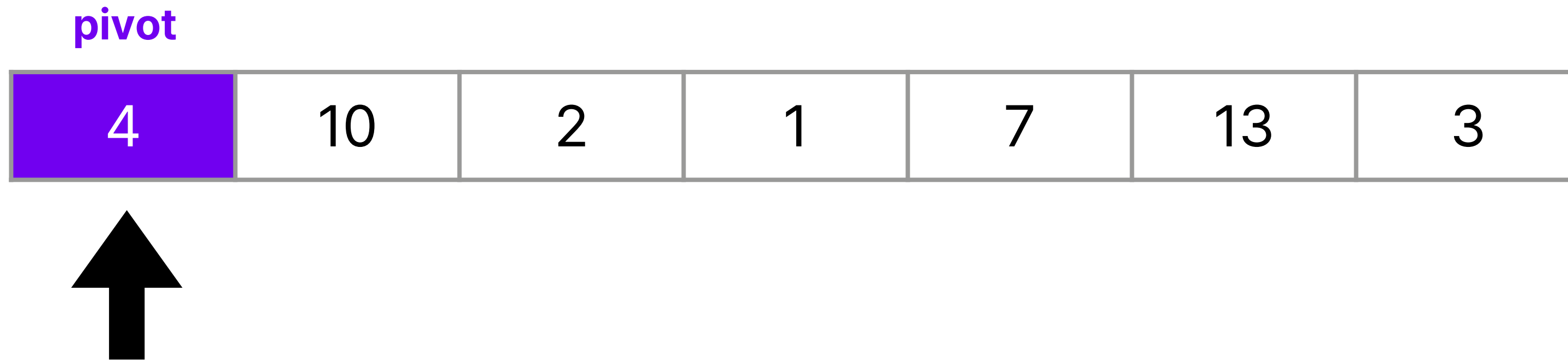
퀵 정렬 (Quick Sort)



퀵 정렬을 위해서는 `array[0]`을 기준으로 설정해야 함
이를 pivot이라고도 부름



퀵 정렬 (Quick Sort)



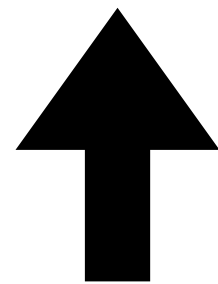
pivot을 기준으로 하나의 큰 리스트를
두 개의 리스트로 나눔



퀵 정렬 (Quick Sort)

pivot

4	10	2	1	7	13	3
---	----	---	---	---	----	---



less_or_equal :

2	1	3
---	---	---

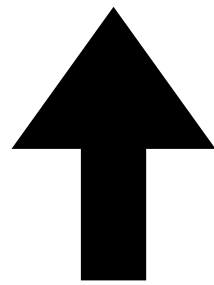
pivot보다 작거나 같은 값들의 리스트(less_or_equal)와



퀵 정렬 (Quick Sort)

pivot

4	10	2	1	7	13	3
---	----	---	---	---	----	---



less_or_equal :

2	1	3
---	---	---

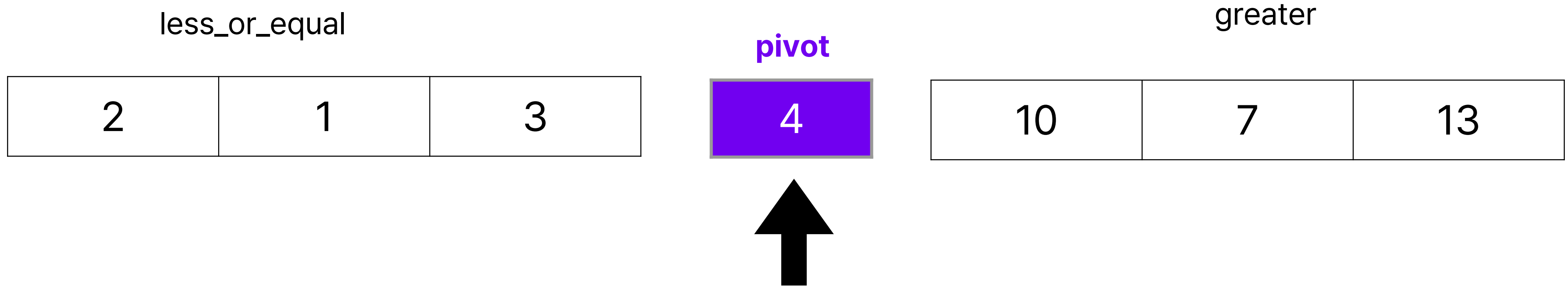
greater :

10	7	13
----	---	----

pivot보다 큰 값들의 리스트(greater)



퀵 정렬 (Quick Sort)

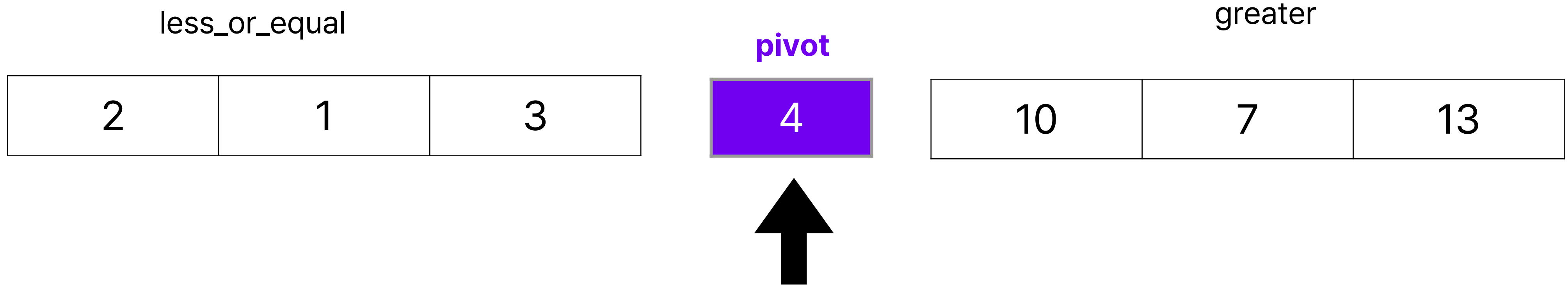


그리고는 less_or_equal, pivot, greater 순서로 리스트를 정렬

아직 리스트가 모두 정렬되지는 않았지만,
4(pivot)에 대해서 정렬이 완료된 것을 확인할 수 있음



퀵 정렬 (Quick Sort)



less_or_equal 리스트와 greater 리스트도
지금까지의 작업을 반복한다면, 정렬이 완성됨



퀵 정렬 (Quick Sort)

less_or_equal

2	1	3
---	---	---

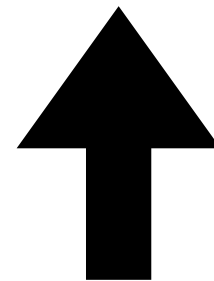
less_or_equal 리스트를 먼저 정렬해보자.



퀵 정렬 (Quick Sort)

pivot

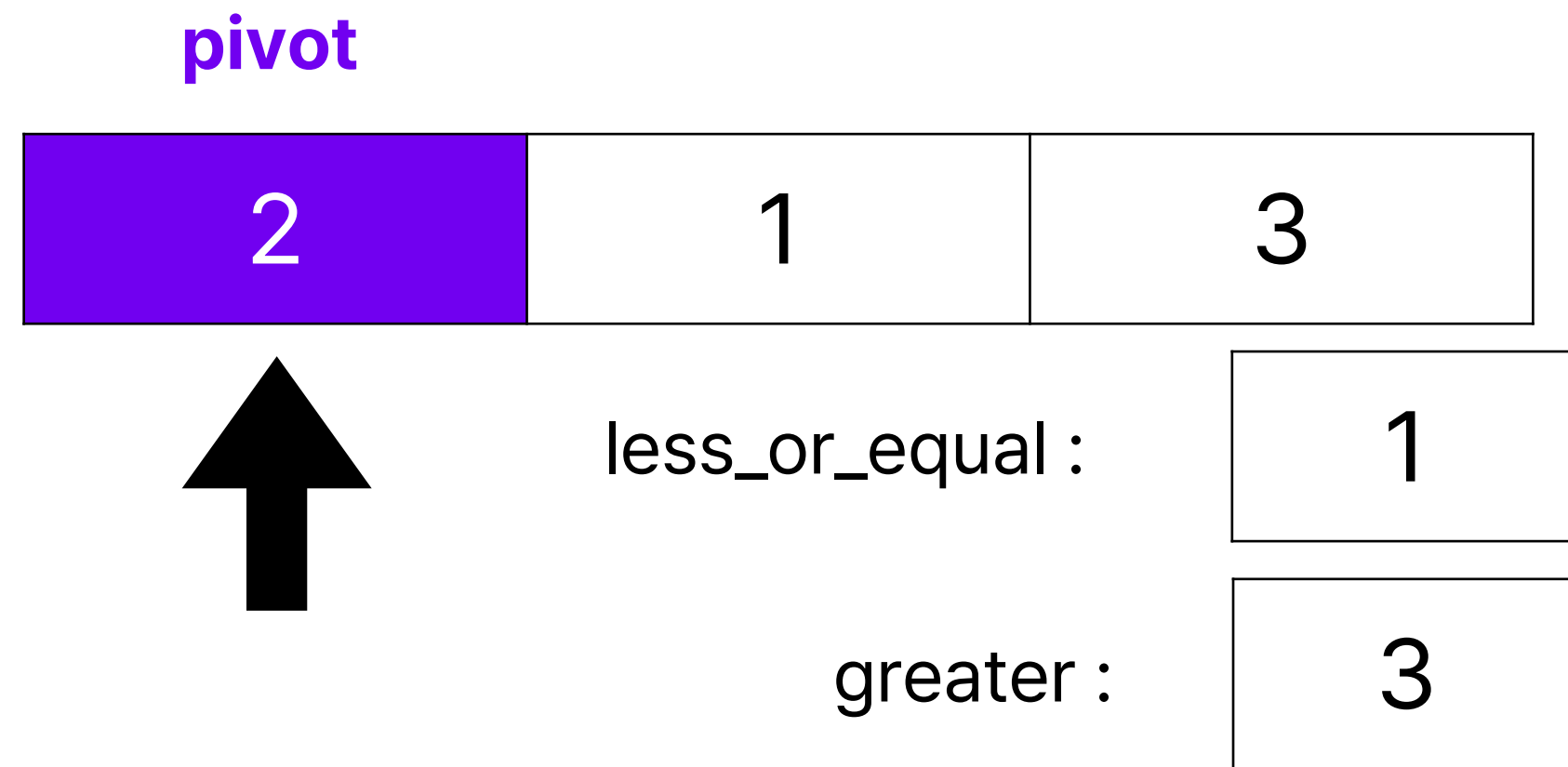
2	1	3
---	---	---



pivot을 설정한 뒤



퀵 정렬 (Quick Sort)

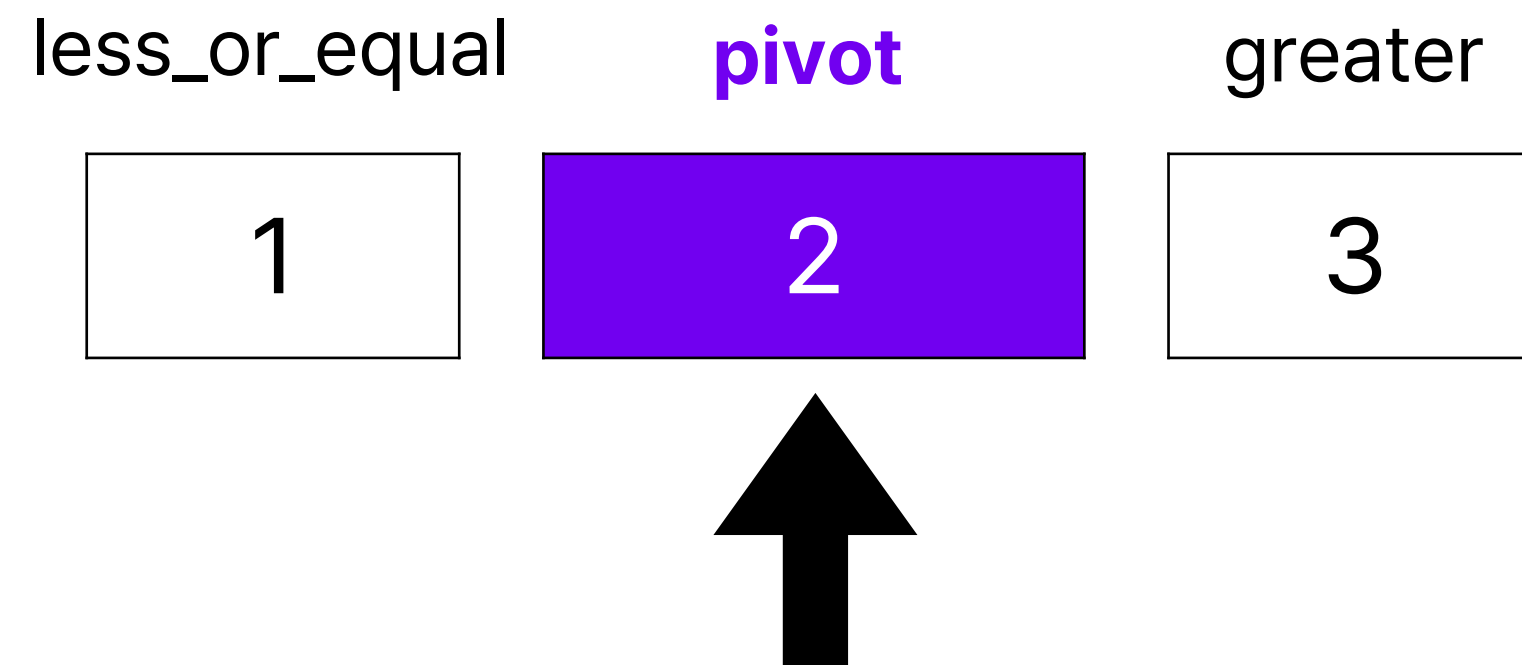


pivot보다 작거나 같은 값들의 리스트(less_or_equal)와

pivot보다 큰 값들의 리스트(greater)로 분할



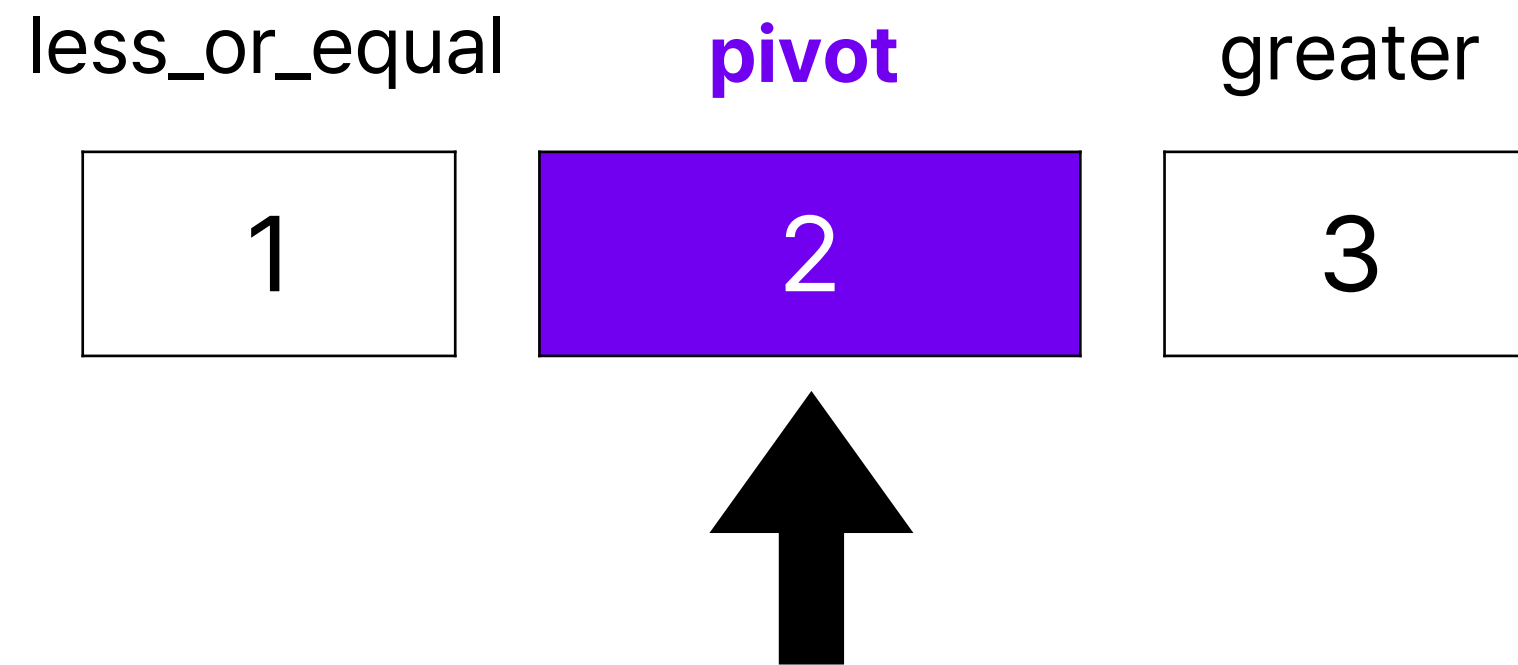
퀵 정렬 (Quick Sort)



그리고는 less_or_equal, pivot, greater 순서로 리스트를 정렬



퀵 정렬 (Quick Sort)



이제 2도 정렬되었음

그뿐만 아니라 less_or_equal 리스트와 greater 리스트로 정렬이 완료됨

분할된 리스트의 길이가 0이거나 1인 경우 더 분할할 필요가 없음



퀵 정렬 (Quick Sort)



원래의 큰 문제 속에서 지금까지의 내용을 합쳐보면,
less_or_equal 리스트 그리고 pivot까지 정렬됨



퀵 정렬 (Quick Sort)



greater 리스트도 정렬해서
전체 리스트의 정렬을 완성해보면?



퀵 정렬 (Quick Sort)

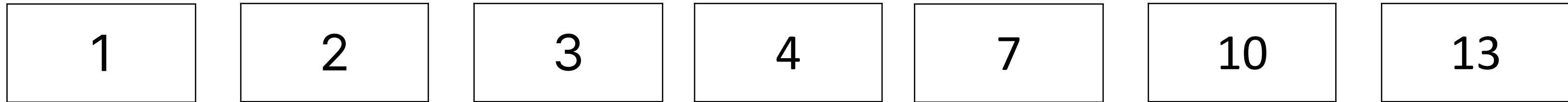


원래의 큰 문제 속에서

작은 문제로 나누어 작업한 결과들을 모두 합쳐보면



퀵 정렬 (Quick Sort)



모든 값이 올바른 순서로 정렬된 것을 확인할 수 있음



퀵 정렬 (Quick Sort)

1. **pivot**을 기준으로 리스트를 분할
2. 리스트의 길이가 **0 or 1**이 될 때까지 반복해서 분할
3. 리스트를 정렬하며 합치기



퀵 정렬 (Quick Sort)

4	7	4	2	10	19	2	4	5	3	1	5
---	---	---	---	----	----	---	---	---	---	---	---

```
def quickSort(data) :  
    if len(data) <= 1 :  
        return data  
  
    pivot = data[0]  
  
    left = getSmallNumbers(data, pivot)  
    right = getLargeNumbers(data, pivot)  
  
    return quickSort(left) + [pivot] + quickSort(right)
```



퀵 정렬 (Quick Sort)

4	7	4	2	10	19	2	4	5	3	1	5
---	---	---	---	----	----	---	---	---	---	---	---

```
def quickSort(data) :  
    if len(data) <= 1 :  
        return data  
  
    → pivot = data[0]  
  
    left = getSmallNumbers(data, pivot)  
    right = getLargeNumbers(data, pivot)  
  
    return quickSort(left) + [pivot] + quickSort(right)
```



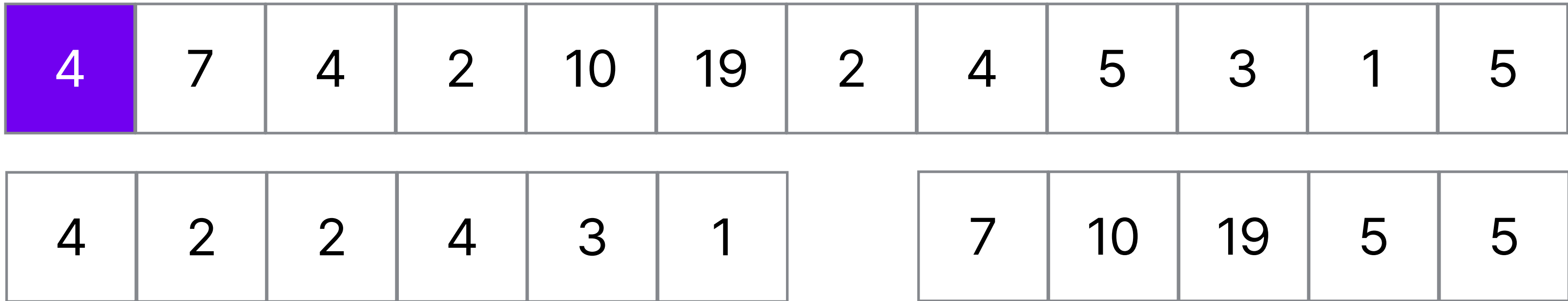

퀵 정렬 (Quick Sort)

4	7	4	2	10	19	2	4	5	3	1	5
4	2	2	4	3	1						

```
def quickSort(data) :  
    if len(data) <= 1 :  
        return data  
  
    pivot = data[0]  
    → left = getSmallNumbers(data, pivot)  
       right = getLargeNumbers(data, pivot)  
  
    return quickSort(left) + [pivot] + quickSort(right)
```



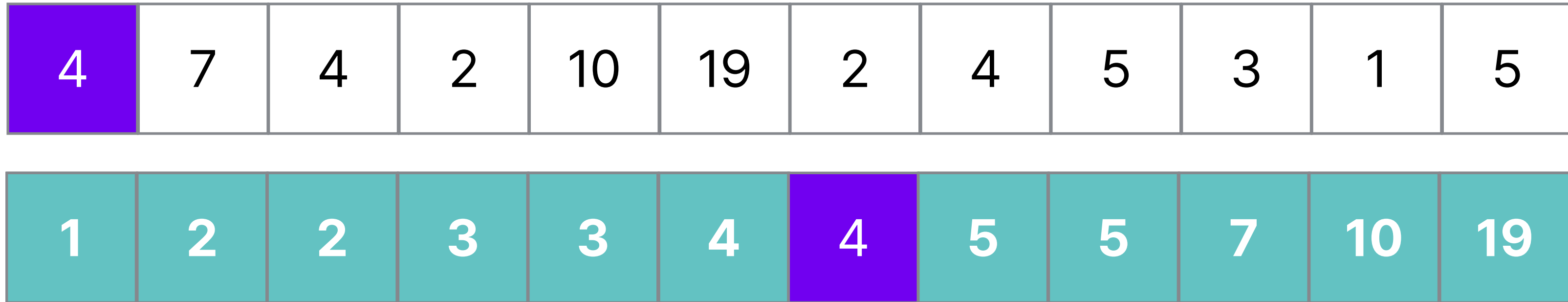
퀵 정렬 (Quick Sort)



```
def quickSort(data) :  
    if len(data) <= 1 :  
        return data  
  
    pivot = data[0]  
  
    → left = getSmallNumbers(data, pivot)  
       right = getLargeNumbers(data, pivot)  
  
    return quickSort(left) + [pivot] + quickSort(right)
```




퀵 정렬 (Quick Sort)



```
def quickSort(data) :  
    if len(data) <= 1 :  
        return data  
  
    pivot = data[0]  
  
    left = getSmallNumbers(data, pivot)  
    right = getLargeNumbers(data, pivot)  
  
    → return quickSort(left) + [pivot] + quickSort(right)
```

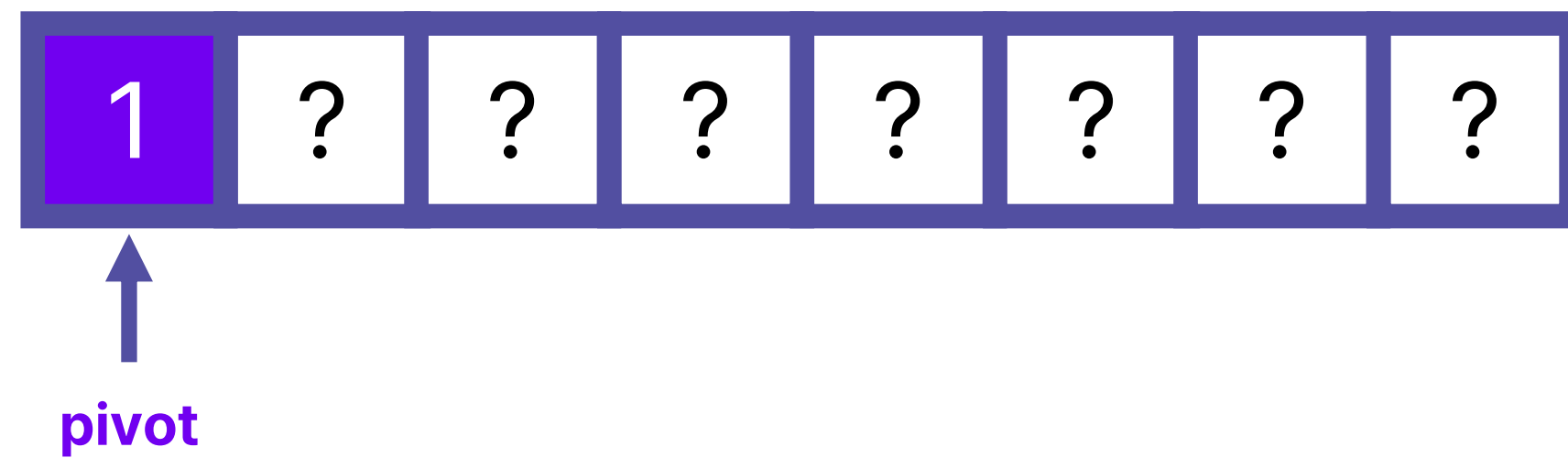


퀵 정렬의 시간 복잡도

퀵 정렬의 시간복잡도는 평균적으로 $O(N\log N)$ 이나,
값에 따라 최악의 경우 $O(N^2)$ 이 될 수 있음



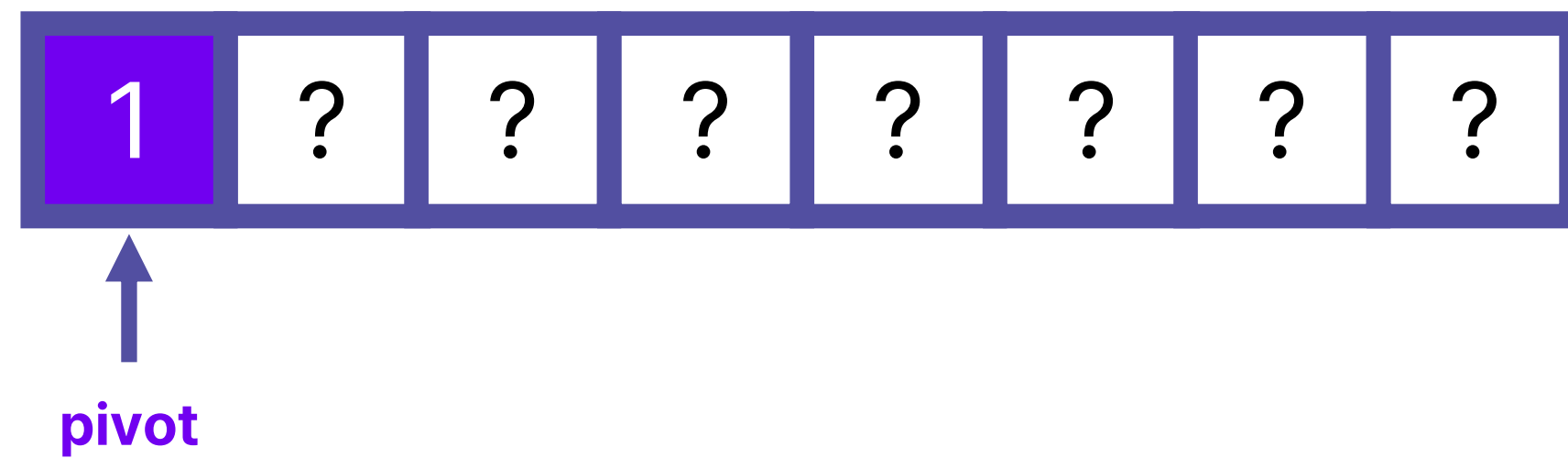
퀵 정렬의 시간 복잡도



만약 맨 앞의 값이 가장 작다면,
값을 분배한 후의 결과는 어떻게 될까?



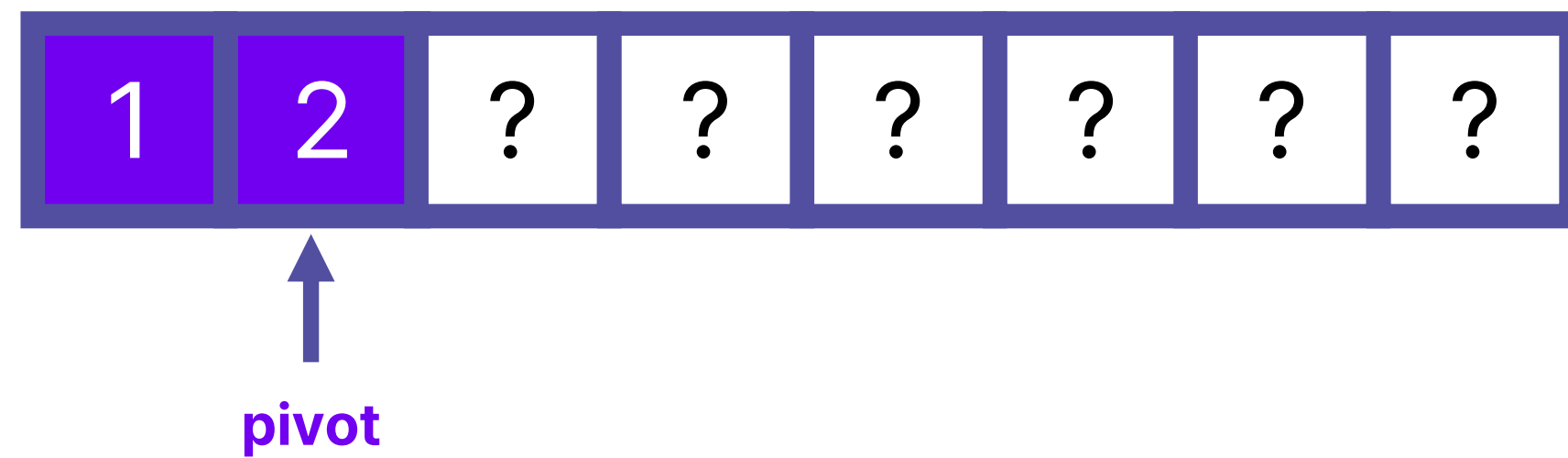
퀵 정렬의 시간 복잡도



당연히 왼쪽에는 아무것도 없고,
오른쪽에 나머지 모든 값이 존재함



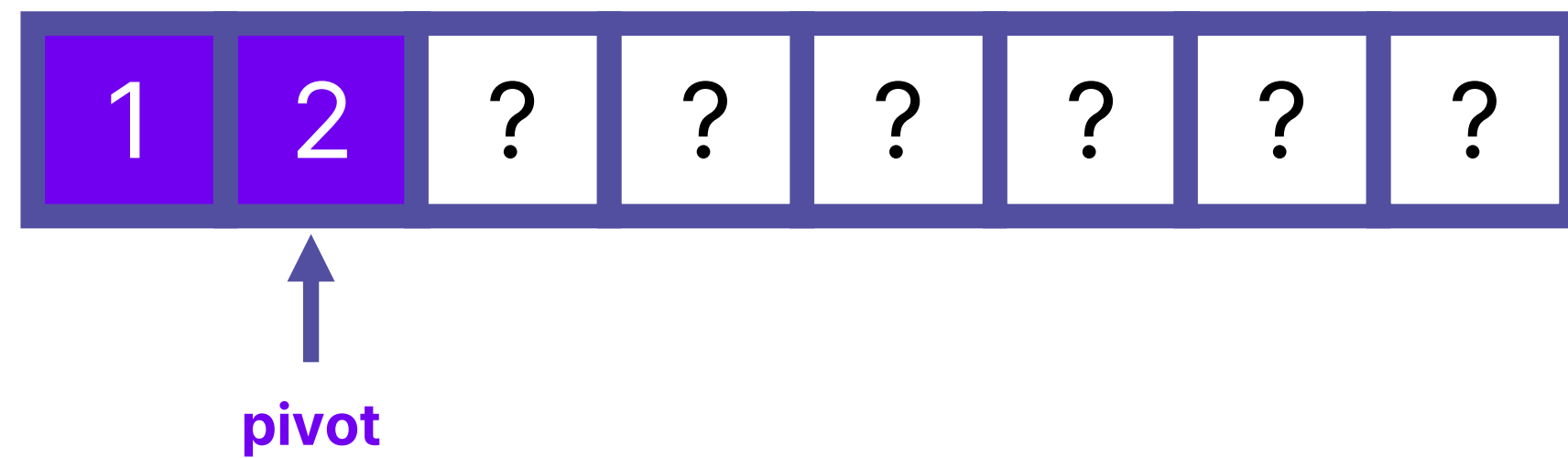
퀵 정렬의 시간 복잡도



그래서 $N - 1$ 개의 원소를 갖고 한 번 더 퀵 정렬을 진행하는데,
여기에서도 맨 앞이 제일 작다면?



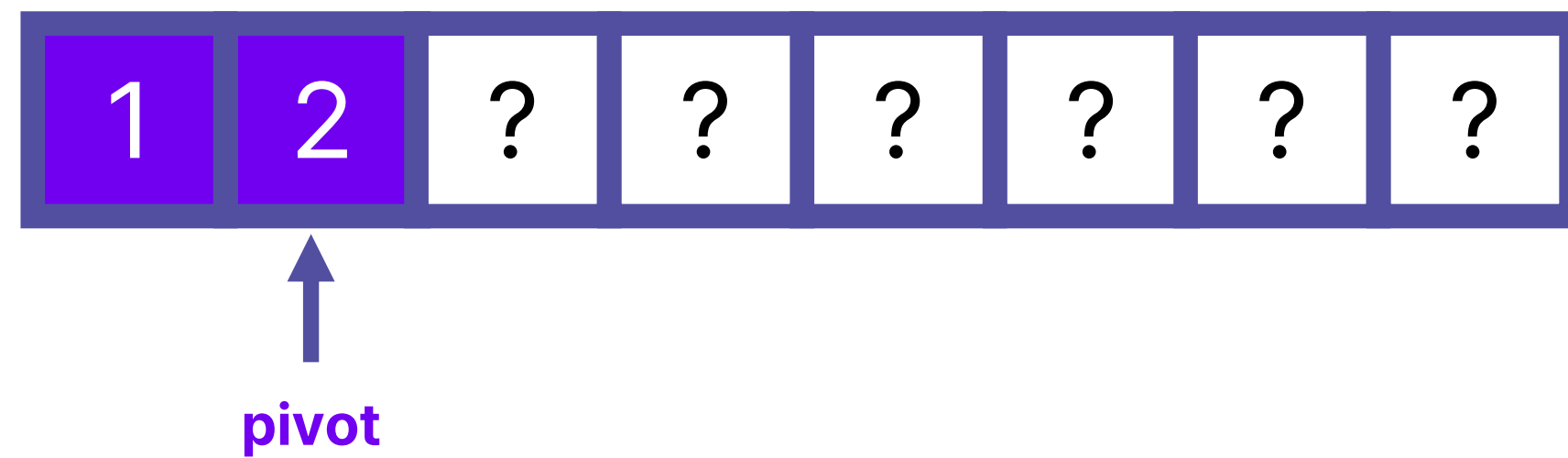
퀵 정렬의 시간 복잡도



계속 이런 과정이 반복된다면,
진행되는 단계의 수는 $\log N$ 이 아니라 N 회가 될 것



퀵 정렬의 시간 복잡도



즉, 이 경우 시간복잡도는 $O(N \log N)$ 이 아니라,
 $O(N^2)$ 이 됨

합병 정렬



합병 정렬 (Merge Sort)

- 정렬되지 않은 리스트를 각각 하나의 원소만 포함하는 n 개의 부분 리스트로 분할
(원소의 개수가 하나인 리스트는 이미 정렬된 것으로 봄)
- 부분 리스트가 하나만 남을 때까지 반복해서 병합하며 정렬된 부분 리스트를 생성
- 마지막 남은 부분 리스트가 정렬된 리스트

6 5 3 1 8 7 2 4



합병 정렬 (Merge Sort)

4	10	2	1	7	13	3
---	----	---	---	---	----	---

합병 정렬(Merge sort)도 퀵 정렬처럼

큰 리스트를 작게 분할하며 빠르게 정렬하는 알고리즘



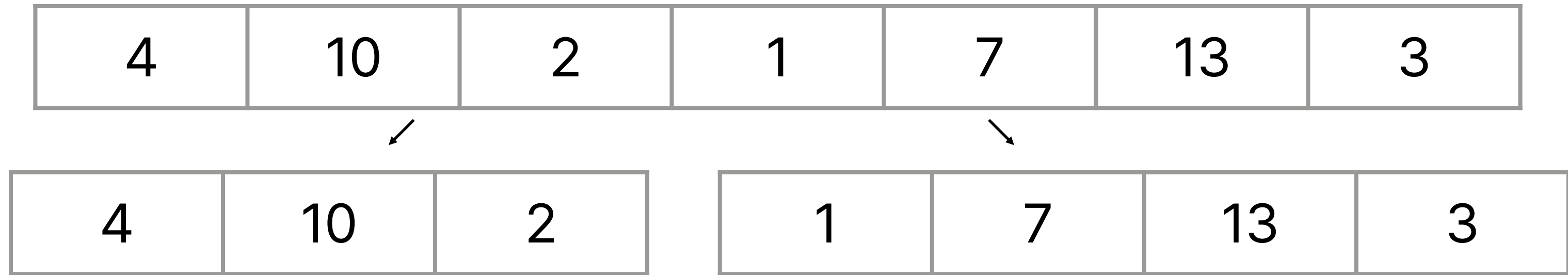
합병 정렬 (Merge Sort)

4	10	2	1	7	13	3
---	----	---	---	---	----	---

합병 정렬(Merge sort)은 값을 비교하지 않고
리스트를 분할함



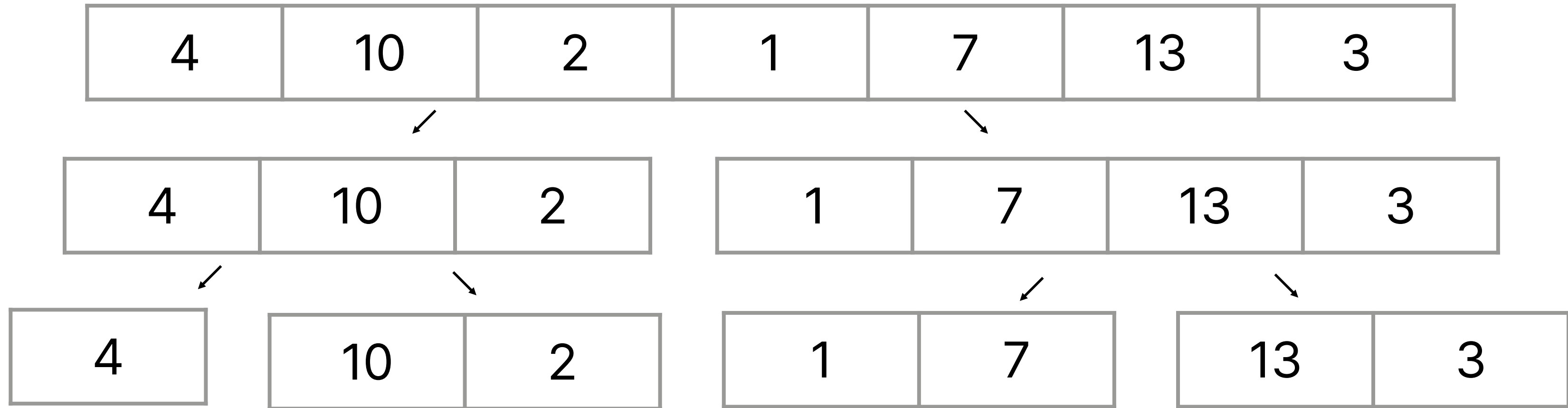
합병 정렬 (Merge Sort)



리스트를 절반으로 나누고



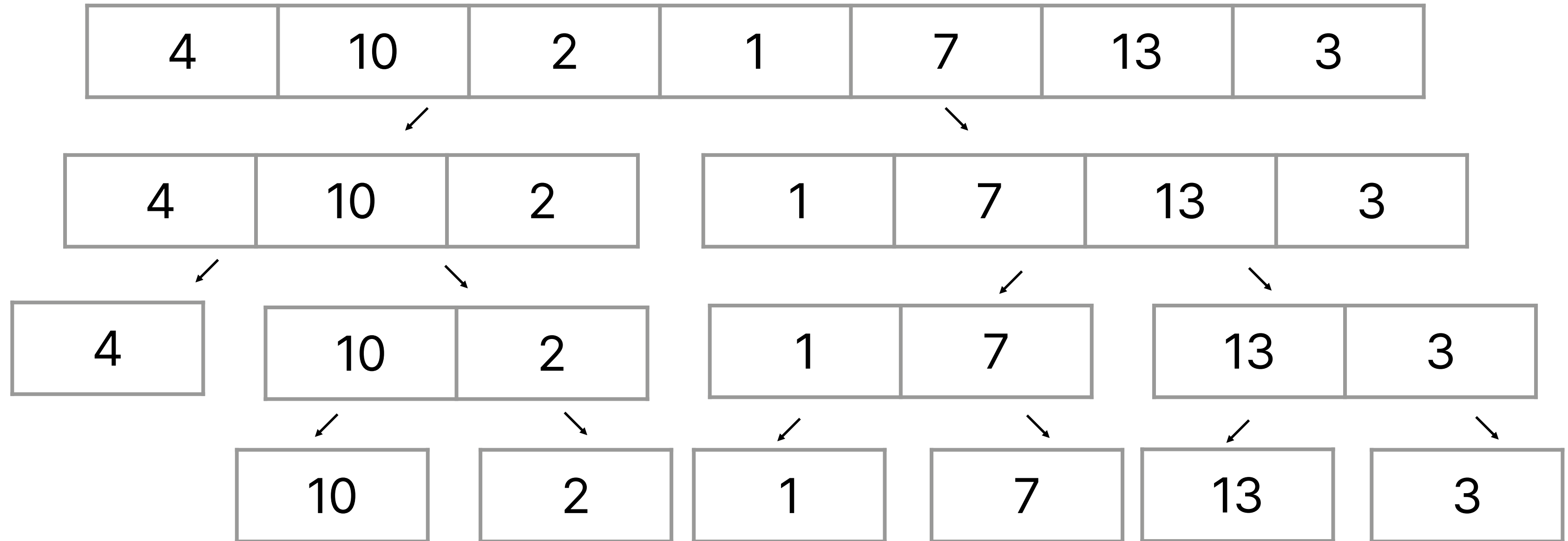
합병 정렬 (Merge Sort)



나눈 리스트를 각각 또 절반으로 나누고



합병 정렬 (Merge Sort)

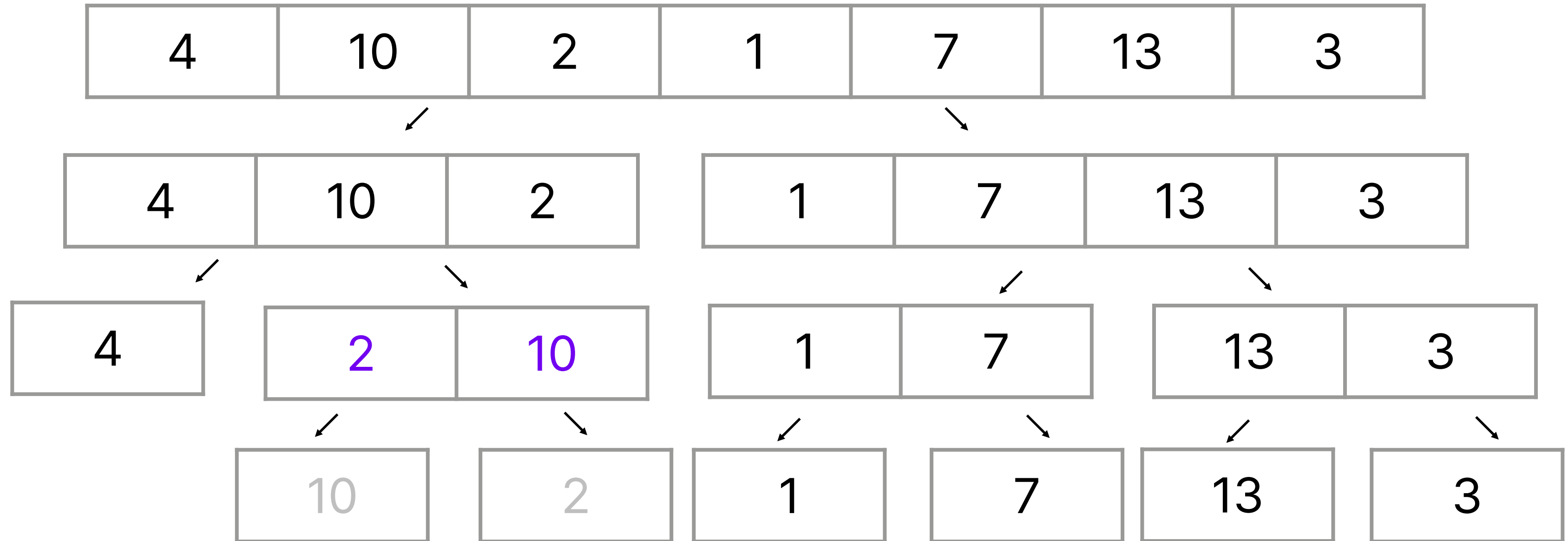


남아있는 리스트의 길이가 0이나 1이면 분할을 멈추고

리스트를 정렬하며 합침



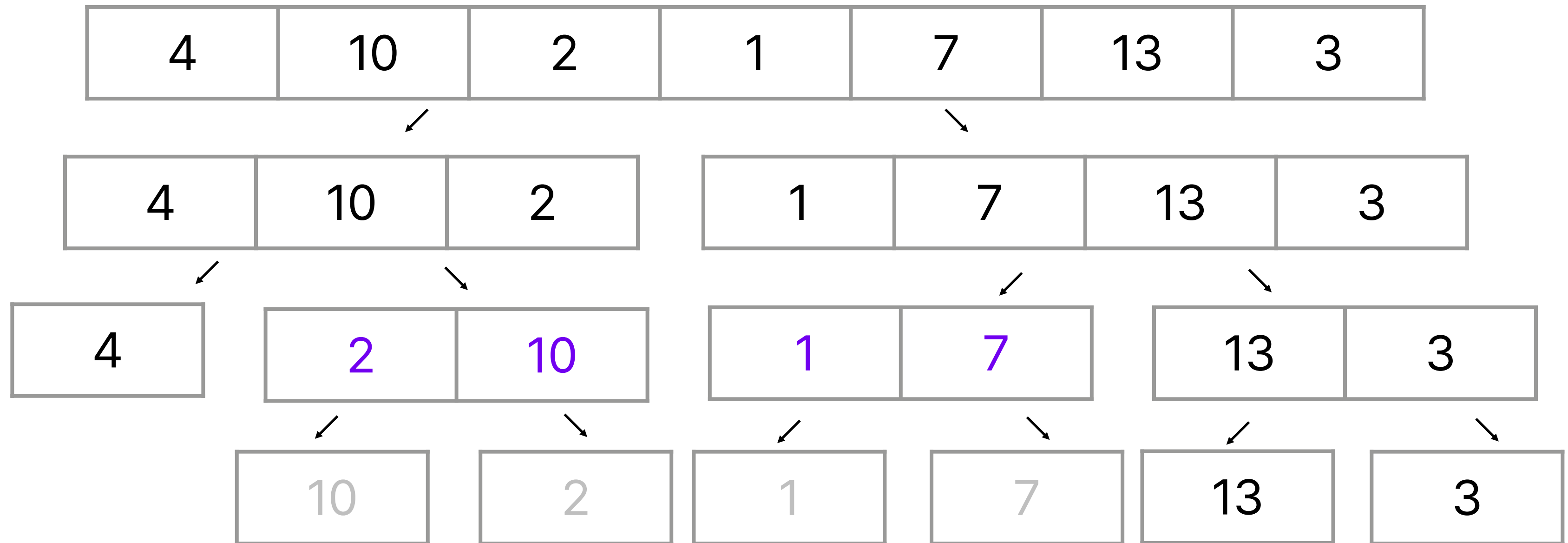
합병 정렬 (Merge Sort)



10과 2를 정렬하고 합치면 [2, 10] 이 되고



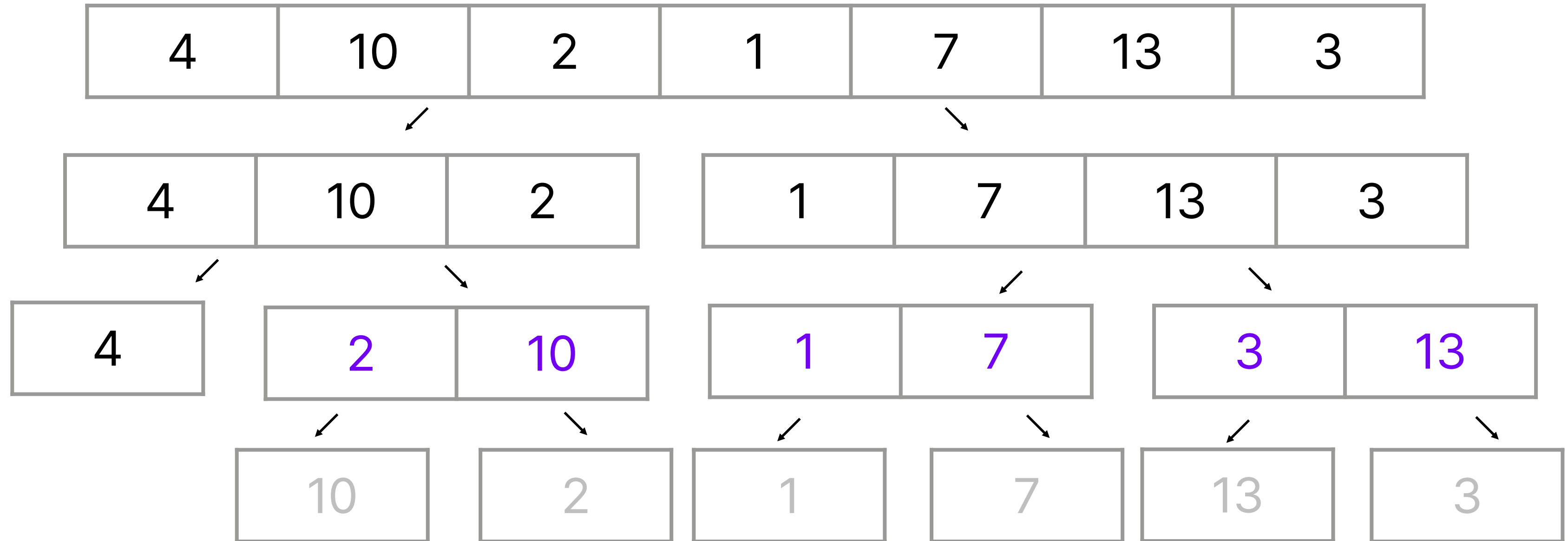
합병 정렬 (Merge Sort)



1과 7을 정렬하고 합치면 [1, 7] 이 되고



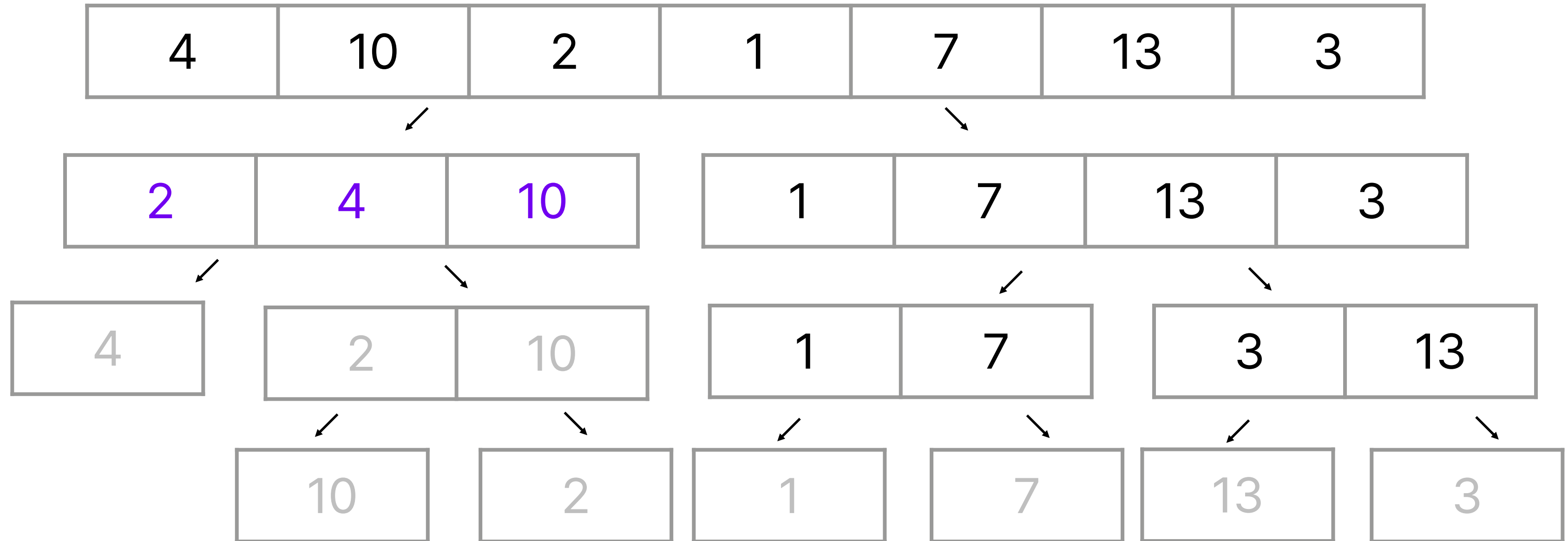
합병 정렬 (Merge Sort)



13과 3을 정렬하고 합쳐서 [3, 13] 이 됨



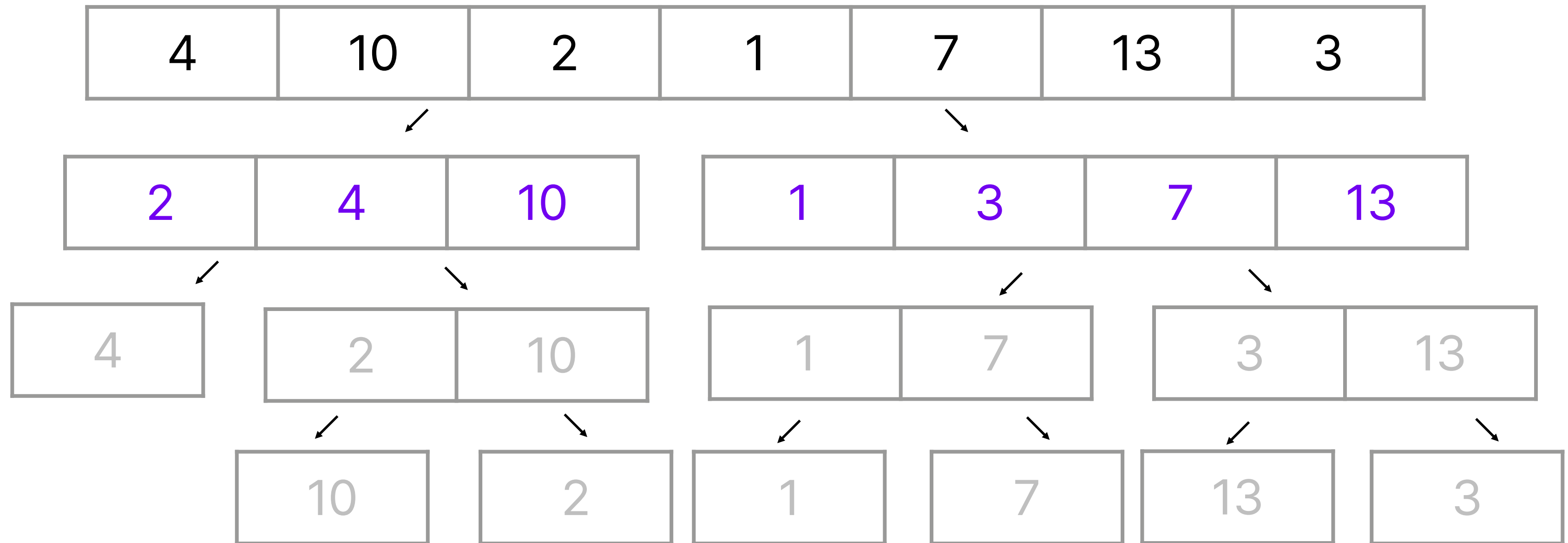
합병 정렬 (Merge Sort)



이 작업을 계속해 올라가다 보면



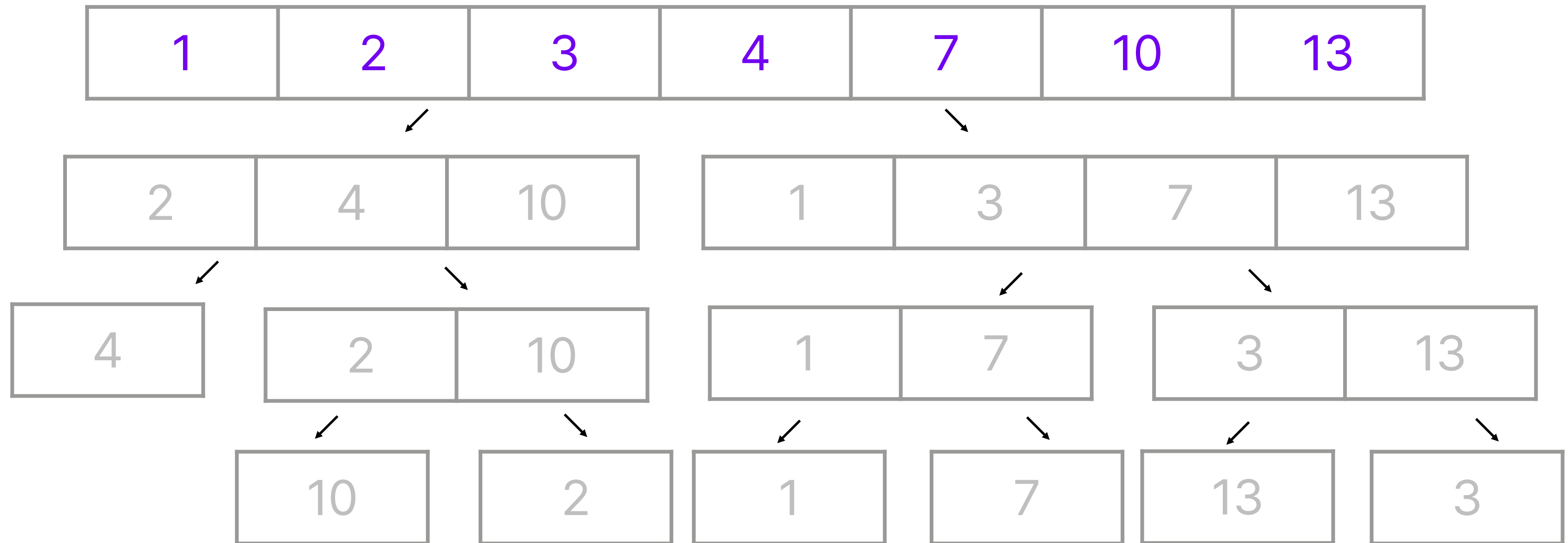
합병 정렬 (Merge Sort)



이 작업을 계속해 올라가다 보면



합병 정렬 (Merge Sort)



처음에 정렬하려고 했던 리스트의 정렬이 완성됨



합병 정렬 (Merge Sort)

1. 리스트를 비교 없이 절반으로 분할
2. 리스트의 길이가 0 or 1이 될 때까지 반복해서 분할
3. 리스트를 정렬하며 합침



퀵 정렬 (Quick Sort) vs. 합병 정렬 (Merge Sort)

퀵 정렬 알고리즘은 **pivot**의 값을 기준으로 분할

합병 정렬 알고리즘은 리스트의 **순서**를 기준으로 분할



$T(N)$: N개의 숫자를 정렬하는 데 걸리는 시간

$$T(N) = 2 * T(N/2) + O(N)$$

3	5	7	2	5	9	13	11	24	11	23	1	4	5	3	2
---	---	---	---	---	---	----	----	----	----	----	---	---	---	---	---



$T(N)$: N개의 숫자를 정렬하는 데 걸리는 시간

3	5	7	2	5	9	13	11	24	11	23	1	4	5	3	2
---	---	---	---	---	---	----	----	----	----	----	---	---	---	---	---

$$T(N) = 2 * T(N/2) + O(N)$$

3	5	7	2	5	9	13	11
---	---	---	---	---	---	----	----

$$T(N/2) = 2 * T(N/4) + O(N/2)$$

3	5	7	2	5	9	13	11
---	---	---	---	---	---	----	----

$$T(N/2) = 2 * T(N/4) + O(N/2)$$



합병 정렬의 시간 복잡도

$T(N)$: N개의 숫자를 정렬하는 데 걸리는 시간

3	5	7	2	5	9	13	11	24	11	23	1	4	5	3	2
---	---	---	---	---	---	----	----	----	----	----	---	---	---	---	---

$$T(N) = 2 * T(N/2) + O(N)$$

$O(N \log N)$

3	5	7	2	5	9	13	11
---	---	---	---	---	---	----	----

3	5	7	2	5	9	13	11
---	---	---	---	---	---	----	----

$$T(N/2) = 2 * T(N/4) + O(N/2)$$

$$T(N/2) = 2 * T(N/4) + O(N/2)$$

3	5	7	2
---	---	---	---

5	9	13	11
---	---	----	----

3	5	7	2
---	---	---	---

5	9	13	11
---	---	----	----

$$T(N/4) = \dots + O(N/4)$$

$$T(N/4) = \dots + O(N/4)$$

$$T(N/4) = \dots + O(N/4)$$

$$T(N/4) = \dots + O(N/4)$$